Equipo 03

Nombre: María José Cedillo Mata **Matrícula**: 610256
Nombre: Alejandra Morón Rdz. **Matrícula**: 587823
Nombre: Fernando Olivares Del Valle **Matrícula**: 600046
Nombre: Ricardo Arath Sánchez Aguirre **Matrícula**: 583928

Proyecto Final. Manual de Uso

Institución educativa: Universidad de Monterrey

Materia: Sistemas Operativos

Profesor: Raúl Morales Salcedo

Gpo: 3305 8:30 am **Semestre:** 6to

Fecha: 11 de mayo de 2026

“Damos nuestra palabra de que hemos realizado esta actividad con integridad académica”

Índice

SetUp Scheduling **4**
Caso 1. FCFS (First Come, First Served) 5
Ejecución 6
Pruebas 7
Imagen 1. Evidencia Visual - First Come First Serve 7
Caso 2. Round Robin (RR) 7
Ejecución 8
Pruebas 9
Imagen 2. Evidencia Visual - Round Robin 9
SetUp Estados **9**
Ejecución 10
Pruebas 11
Imagen 3. Estados de Procesos 11
SetUp Multicore y Paralelismo **11**
Caso 1. Configuración con 1 Core 12
Ejecución 12
Pruebas 13
Imagen 4. Configuración con 1 Core 13
Caso 2. Configuración con 2 cores 13
Ejecución 13
Pruebas 14
Imagen 5. Configuración con 2 Cores 14
Caso 3. Configuración con 4 cores 15
Ejecución 15
Pruebas 15
Imagen 5. Configuración con 2 Cores 15
SetUp Reemplazo de Páginas **16**
Caso 1. First-In, Frist-Out 16
Ejecución 16
Resultados Observados (FIFO): 16
Pruebas 17
Imagen 7. FIFO 17
Caso 2. LRU 18
Ejecución 18
Pruebas 18
Imagen 8. LRU 18
Caso 3. Óptimo 19
Ejecución 19
Pruebas 19
Imagen 9. Óptimo 19
SetUp Threads **20**

Caso 1. Many-to-One.....20
Ejecución.....20
Pruebas.....21
Imagen 10. Many to One.....21
Caso 2. One-to-One.....22
Ejecución.....22
Pruebas.....22
Imagen 11. One to One.....22
Caso 3. Many-to-Many.....23
Ejecución.....23
Pruebas.....24
Imagen 12. Many to Many.....24
SetUp Comparación de Algoritmos.....24
Caso 1. FCFS vs SJF vs Round Robin vs Priority vs HRRN.....26
Ejecución.....26
Pruebas.....26
Imagen 1. FCFS vs SJF vs Round Robin vs Priority vs HRRN.....26

SetUp Scheduling

Objetivo: Comparar el comportamiento y desempeño de los algoritmos de planificación First Come, First Served (FCFS) y Round Robin (RR), analizando el impacto del uso de planificación apropiativa y no apropiativa sobre métricas como tiempo de espera, turnaround time, response time y cantidad de cambios de contexto.

Configuración del Escenario:

- Módulo utilizado: scheduling.html
- Cantidad de procesos: 5 procesos (P1 a P5)
- Variable independiente:
    - Algoritmo de planificación
        - FCFS (Non-Preemptive)
        - Round Robin (Preemptive)
- Quantum utilizado en Round Robin: 4 ms
- Burst Times: variables entre 2 ms y 9 ms

Datos de Entrada:

Proceso	Arrival Time (AT)	Burst Time (BT)
P1	0	8
P2	1	4
P3	2	9
P4	3	5
P5	4	2

Caso 1. FCFS (First Come, First Served)

Descripción del Algoritmo: El algoritmo FCFS utiliza una política de atención basada en orden de llegada. El primer proceso que entra al sistema es el primero en recibir el CPU y mantiene el control del procesador hasta finalizar completamente su ejecución. Debido a que no permite interrupciones, se clasifica como un algoritmo Non-Preemptive. Conceptualmente, este comportamiento puede compararse con una fila tradicional de atención, donde cada proceso debe esperar a que todos los anteriores concluyan antes de poder ejecutarse.

Métricas Globales (FCFS)

Métrica	Resultado
Avg Turnaround	17.0 ms
Avg Waiting	11.4 ms
Avg Response	11.4 ms
CPU Utilization	100%
Context Switches	4

Resultados por Proceso (FCFS)

PID	Arrival	Burst	Completition	Turnaround	Waiting	Response
P1	0	8	8	8	0	0
P2	1	4	12	11	7	7
P3	2	9	21	19	10	10
P4	3	5	26	23	18	18
P5	4	2	28	24	22	22

Comportamiento Observado (FCFS): Durante la simulación se observó que los procesos fueron ejecutados estrictamente en el orden en que llegaron al sistema. El proceso P1 monopolizó inicialmente el CPU debido a su ráfaga relativamente larga,

obligando a los procesos posteriores a permanecer en espera hasta que concluyera su ejecución. Este comportamiento provocó el denominado Convoy Effect, donde procesos cortos como P5 experimentaron tiempos de espera excesivos a pesar de requerir muy poco tiempo de CPU. P5, por ejemplo, necesitaba únicamente 2 ms de ejecución, pero tuvo que esperar 22 ms antes de comenzar. Asimismo, el algoritmo presentó una baja cantidad de cambios de contexto debido a que únicamente ocurre un cambio cuando un proceso finaliza completamente. Esto reduce el overhead del sistema, aunque sacrifica capacidad de respuesta.

Ejecución

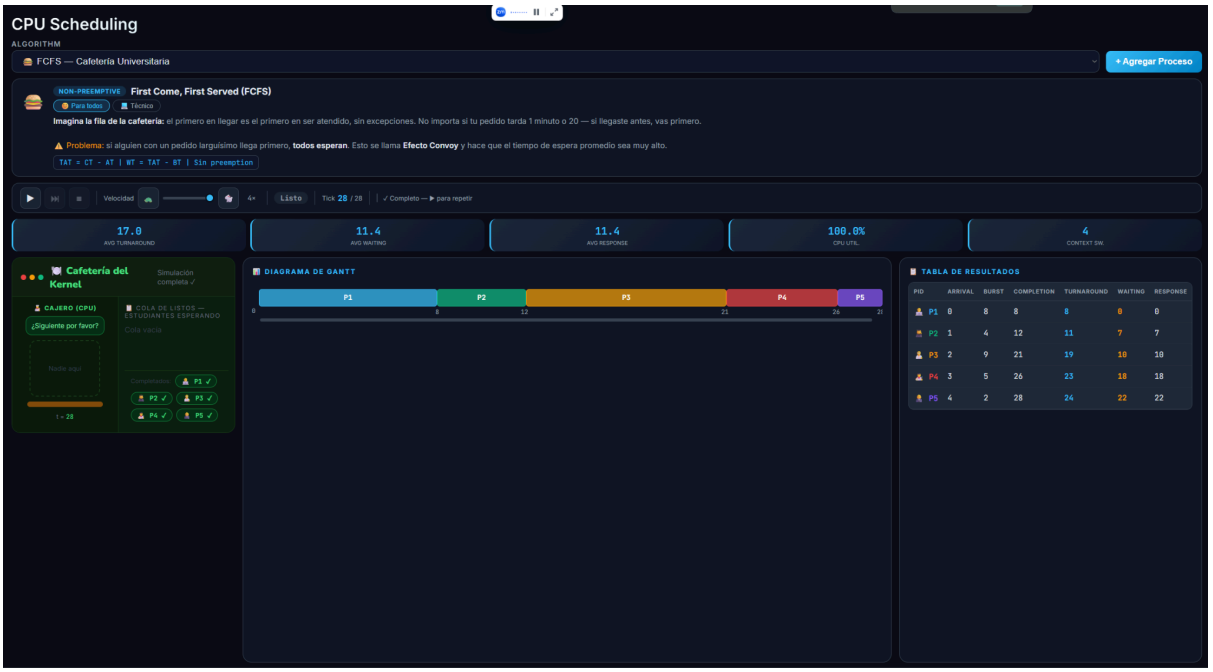
Resultados por Proceso (FCFS)

PID	Arrival	Burst	Completition	Turnaround	Waiting	Response
P1	0	8	8	8	0	0
P2	1	4	12	11	7	7
P3	2	9	21	19	10	10
P4	3	5	26	23	18	18
P5	4	2	28	24	22	22

Comportamiento Observado (FCFS): Durante la simulación se observó que los procesos fueron ejecutados estrictamente en el orden en que llegaron al sistema. El proceso P1 monopolizó inicialmente el CPU debido a su ráfaga relativamente larga, obligando a los procesos posteriores a permanecer en espera hasta que concluyera su ejecución. Este comportamiento provocó el denominado Convoy Effect, donde procesos cortos como P5 experimentaron tiempos de espera excesivos a pesar de requerir muy poco tiempo de CPU. P5, por ejemplo, necesitaba únicamente 2 ms de ejecución, pero tuvo que esperar 22 ms antes de comenzar. Asimismo, el algoritmo presentó una baja cantidad de cambios de contexto debido a que únicamente ocurre un cambio cuando un proceso finaliza completamente. Esto reduce el overhead del sistema, aunque sacrifica capacidad de respuesta.

Pruebas

Imagen 1. Evidencia Visual - First Come First Serve



Comentarios. Al trabajar el algoritmo FCFS con los arrival time como prioridades, es posible observar un resultado adecuado al haber ordenado en el diagrama de gantt los procesos con arrival time ascendente. Es decir, de menor a mayor.

Caso 2. Round Robin (RR)

Descripción del Algoritmo: El algoritmo Round Robin utiliza planificación apropiativa (Preemptive Scheduling) basada en un quantum de tiempo. Cada proceso recibe un intervalo máximo de ejecución; si no concluye dentro de dicho periodo, es interrumpido y enviado nuevamente al final de la cola de listos. Este enfoque distribuye el CPU de manera más equitativa entre todos los procesos y es ampliamente utilizado en sistemas operativos interactivos debido a su buena capacidad de respuesta.

Configuración del Quantum Quantum

- Quantum = 4 ms

Métricas Globales (RR):

Métrica	Resultado
Avg Turnaround	16.5 ms
Avg Waiting	11.0 ms
Avg Response	4.8 ms
CPU Utilization	100%
Context Switches	8

Ejecución

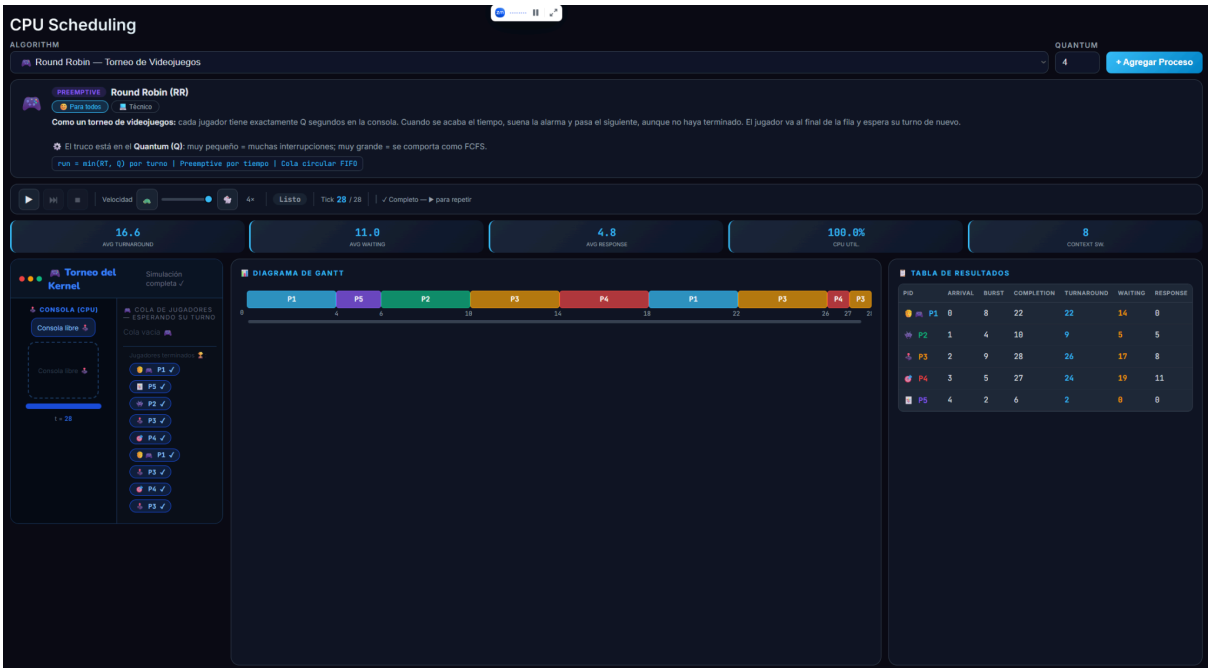
Resultados por Proceso (RR):

PID	Arrival	Burst	Completion	Turnaround	Waiting	Response
P1	0	8	22	22	14	0
P2	1	4	10	9	5	5
P3	2	9	28	26	17	8
P4	3	5	27	24	19	11
P5	4	2	6	2	0	0

Comportamiento Observado (Round Robin): Durante la ejecución de Round Robin se observó una distribución mucho más equitativa del CPU entre todos los procesos. Gracias al uso del quantum, ningún proceso monopolizó completamente el procesador, permitiendo que incluso procesos cortos como P5 fueran atendidos rápidamente. El tiempo de respuesta promedio disminuyó significativamente en comparación con FCFS, pasando de 11.4 ms a 4.8 ms. Esto indica que los procesos comenzaron a ejecutarse mucho antes después de llegar al sistema, mejorando considerablemente la percepción de rapidez y capacidad de respuesta. Sin embargo, esta mejora implicó un aumento considerable en la cantidad de cambios de contexto. Cada vez que un proceso agotaba su quantum sin terminar, el sistema debía guardar su estado y cargar otro proceso, incrementando el overhead de planificación.

Pruebas

Imagen 2. Evidencia Visual - Round Robin



Comentarios. Para el algoritmo RR se puede visualizar en la imagen el uso de la variable de quantum time como tiempo máximo para ejecutar un proceso si aún hay más en la cola de listos. Se ve en el diagrama de gantt que el primer proceso en llegar a la CPU fue P1, asimismo el primero en empezar su ejecución. Al ser un algoritmo preemptive, observamos que se interrumpieron procesos a lo largo de la ejecución antes de terminarlos todos.

SetUp Estados

Objetivo: Analizar el comportamiento del ciclo de vida de los procesos dentro del simulador, observando las transiciones entre estados, la gestión de procesos bloqueados y la representación visual de la concurrencia en tiempo real.

Configuración del Escenario:

- Módulo utilizado: index.html
- Cantidad de procesos: 3 procesos
- Variables observadas:
    - Estados del proceso
    - Transiciones entre estados
    - Uso del estado *Waiting*
    - Flujo visual de ejecución

- Actualización de métricas en tiempo real

Estados Identificados: Durante la ejecución del escenario, el simulador mostró los cinco estados clásicos del modelo de procesos de sistemas operativos:

- NEW (Nuevo)
- READY (Listo)
- RUNNING (En ejecución)
- WAITING (Esperando)
- TERMINATED (Terminado)

Estos estados fueron visualizados dentro del panel denominado *State Diagram*, donde el sistema representa dinámicamente el recorrido de cada proceso a lo largo de su ciclo de vida.

Datos de Entrada:

Proceso	Arrival Time	Burst Time	Priority	Pages
P5	0 ms	5 ms	2	3
P896	2 ms	6 ms	2	1
P455	1 ms	6 ms	1	6

Ejecución

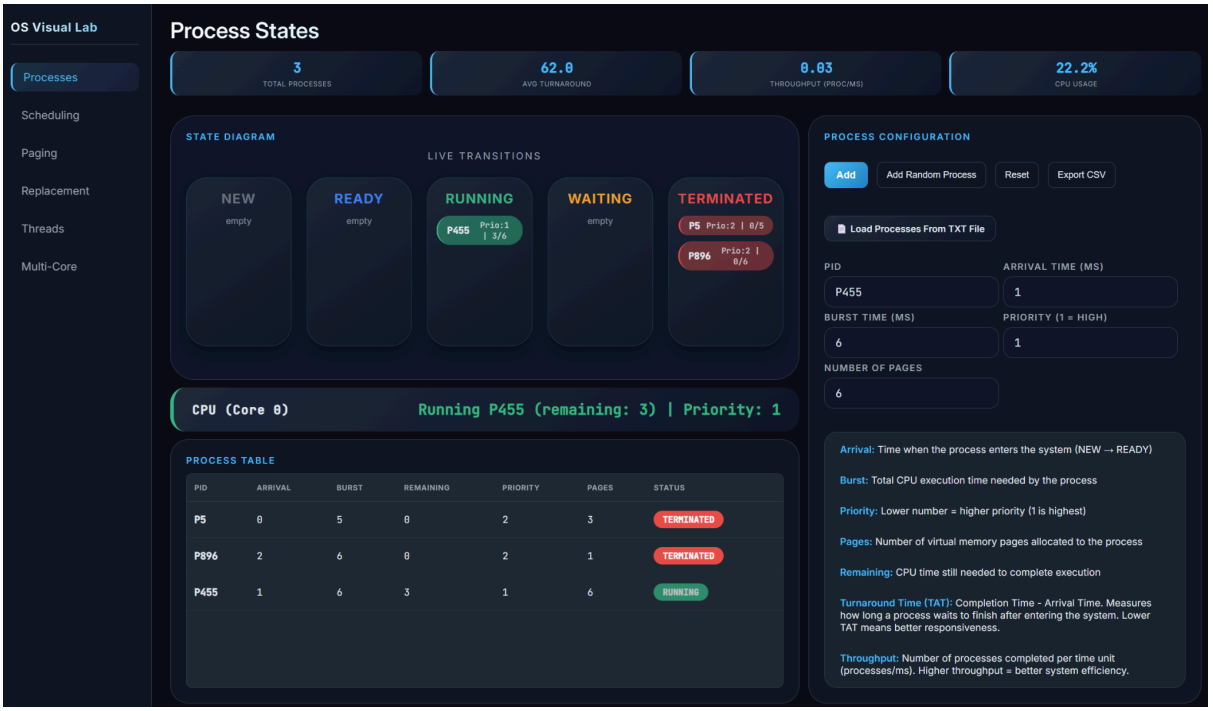
Resultados del Escenario

Proceso	Arrival Time	Burst Time	Priority	Pages	Estado Final
P5	0 ms	5 ms	2	3	Terminated
P896	2 ms	6 ms	2	1	Terminated
P455	1 ms	6 ms	1	6	Terminated

Los resultados obtenidos permitieron comprobar que el simulador representa correctamente el modelo clásico de estados de procesos utilizado en sistemas operativos. La simulación mostró transiciones coherentes entre estados, incluyendo escenarios de bloqueo y reanudación de ejecución. Asimismo, la representación visual animada facilitó la comprensión de la concurrencia y del comportamiento dinámico del scheduler, permitiendo observar cómo múltiples procesos evolucionan de manera intercalada dentro del sistema.

Pruebas

Imagen 3. Estados de Procesos



Comentarios. Los procesos pasan por cada fase como se había detallado. Estos pasan el largo de su tiempo de burst en estado Running y pasan al final a Terminated. Las métricas muestran un Turnaround promedio de 62 y un Throughout de 0.03 procesos/ms

SetUp Multicore y Paralelismo

Objetivo: Analizar el impacto del número de núcleos de procesamiento sobre el tiempo total de ejecución, la utilización del CPU y la distribución de carga entre cores dentro de la simulación multicore del sistema.

Configuración del Escenario:

- Módulo utilizado: multicore.html
- Cantidad de hilos simulados:
    - T1 = 3 ms
    - T2 = 2 ms

- Configuraciones evaluadas:
    - 1 Core
    - 2 Cores
    - 4 Cores
- Variables observadas:
    - Tiempo total de ejecución
    - Utilización del CPU
    - Distribución de carga
    - Paralelismo visual
    - Turnaround promedio

Caso 1. Configuración con 1 Core

Ejecución

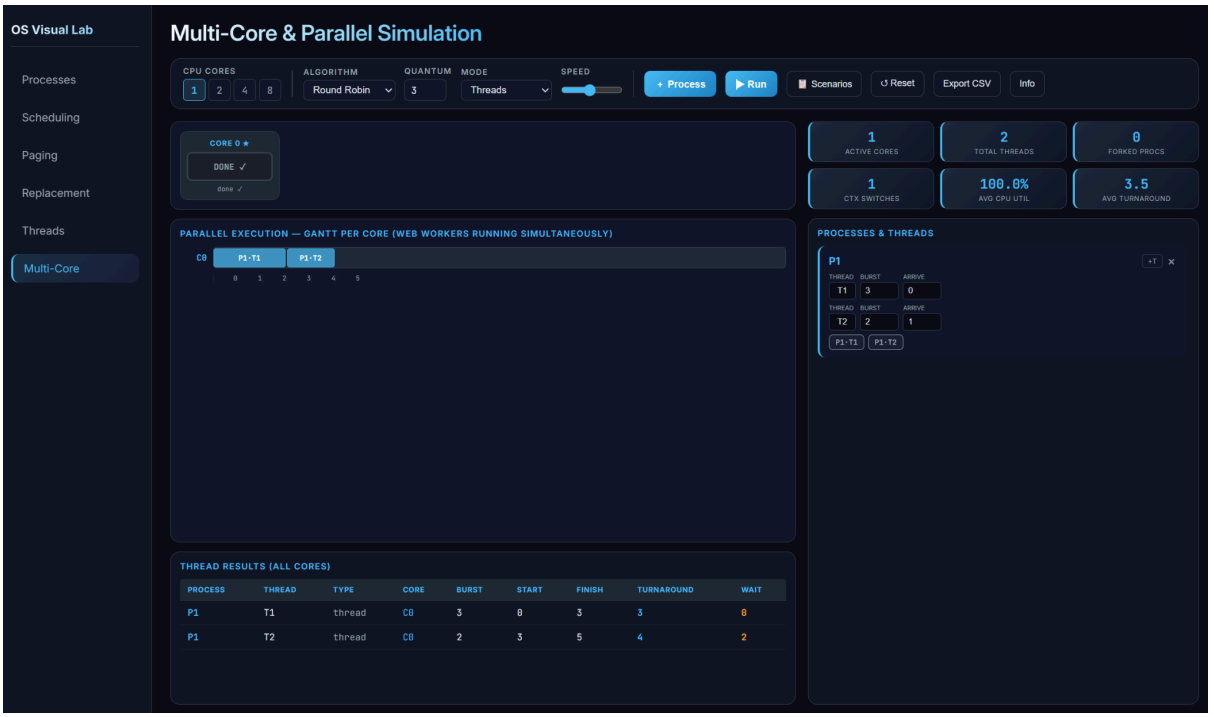
Resultados Observados:

Métrica	**Resultado**
Tiempo total de ejecución	5 ticks
Avg Turnaround	3.5
CPU Utilization	100%

Análisis: La configuración de un solo núcleo presentó una utilización máxima del CPU, ya que el único core permaneció ocupado durante toda la simulación. Sin embargo, la ausencia de paralelismo provocó un mayor tiempo total de ejecución y tiempos de espera adicionales para los hilos posteriores.

Pruebas

Imagen 4. Configuración con 1 Core



Comentarios: Durante la simulación con un único núcleo de procesamiento, ambos hilos fueron ejecutados de manera secuencial. El Core 0 ejecutó primero el hilo T1 y posteriormente T2, debido a que no existía otra unidad de procesamiento disponible para realizar una ejecución paralela.

Caso 2. Configuración con 2 cores

Ejecución

Resultados Observados

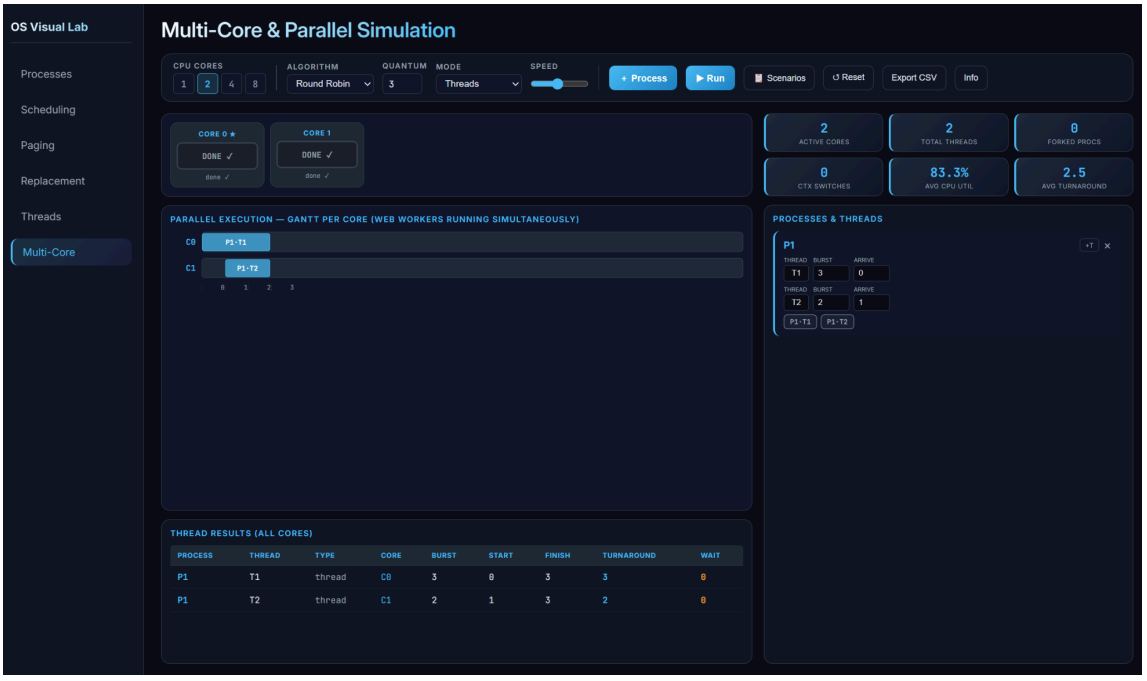
Métrica	**Resultado**
Tiempo total de ejecución	3 ticks
Avg Turnaround	2.5
CPU Utilization	83.3%

Análisis: La incorporación de un segundo núcleo produjo una mejora significativa en el desempeño general del sistema. El tiempo total de ejecución se redujo considerablemente debido a la capacidad de ejecutar múltiples hilos de manera paralela.

Asimismo, desaparecieron los tiempos de espera observados en la configuración de un solo núcleo, ya que ambos hilos pudieron acceder al CPU simultáneamente. Esto mejoró el turnaround promedio y generó una percepción visual de mayor velocidad y eficiencia.

Pruebas

Imagen 5. Configuración con 2 Cores



Comentarios. Al utilizar dos núcleos de procesamiento, el scheduler distribuyó los hilos entre ambos cores, asignando T1 al Core 0 y T2 al Core 1. Esta distribución permitió la ejecución concurrente de ambas tareas.

Visualmente, el diagrama de Gantt mostró barras superpuestas verticalmente, indicando que los hilos se ejecutaban al mismo tiempo dentro de distintos núcleos. El tiempo total de ejecución quedó determinado por el hilo más largo, correspondiente a T1.

Caso 3. Configuración con 4 cores

Ejecución

Resultados Observados

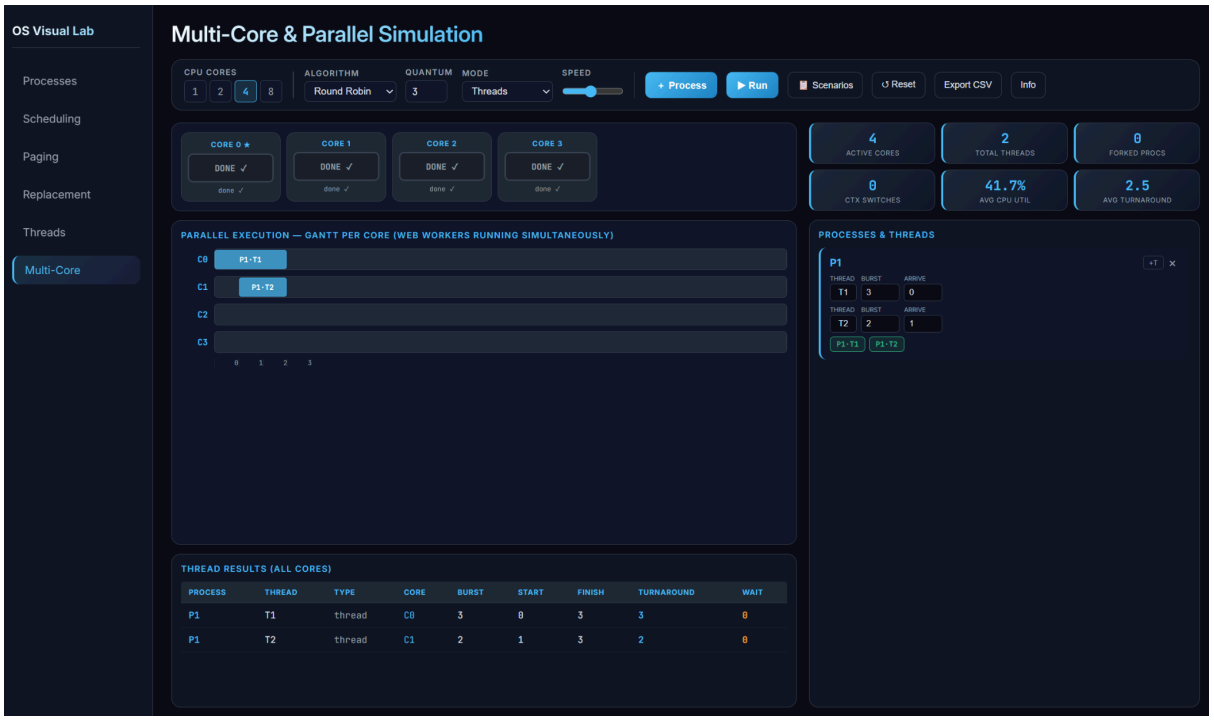
Métrica	Resultado
Tiempo Total de ejecución	3 ticks
Average Turnaround	2.5
CPU Utilization	41.7%

Análisis: Aunque la cantidad de núcleos aumentó, el sistema no obtuvo beneficios adicionales en desempeño debido a que la carga de trabajo era insuficiente para aprovechar completamente los recursos disponibles.

La reducción en la utilización promedio del CPU evidencia la existencia de núcleos ociosos durante gran parte de la simulación. Esto demuestra que incrementar el número de cores no garantiza automáticamente mejoras de rendimiento si la cantidad de tareas concurrentes es limitada.

Pruebas

Imagen 5. Configuración con 2 Cores



Comentarios: En la configuración con cuatro núcleos, únicamente dos cores fueron utilizados activamente debido a que el número de hilos disponibles era menor al número de núcleos existentes. Los Core 0 y Core 1 ejecutaron los hilos T1 y T2 respectivamente, mientras que Core 2 y Core 3 permanecieron inactivos durante toda la simulación.

El diagrama de Gantt mostró nuevamente ejecución paralela entre los dos hilos disponibles, pero no se observaron mejoras adicionales en el tiempo total de ejecución con respecto a la configuración de dos núcleos.

SetUp Reemplazo de Páginas

Objetivo: Comparar el comportamiento y eficiencia de los algoritmos de reemplazo de páginas FIFO, LRU y Óptimo mediante el análisis de fallos de página, aciertos de memoria y administración de frames dentro de un entorno de memoria virtual simulada.

Configuración del Escenario

- Módulo utilizado: replacement.html
- Cantidad de frames: 3
- Secuencia de referencias utilizada:
    - 7, 0, 1, 2, 0, 3, 0, 4.
- Algoritmos evaluados:
    - FIFO (First-In, First-Out)
    - LRU (Least Recently Used)
    - Óptimo (Optimal / Belady)
- Variables observadas:
    - Número de Page Faults
    - Cantidad de Hits
    - Hit Ratio
    - Comportamiento de reemplazo
    - Estado final de los frames

Caso 1. First-In, Frist-Out

Ejecución

Resultados Observados (FIFO):

Métrica	**Resultado**
Page Faults	7

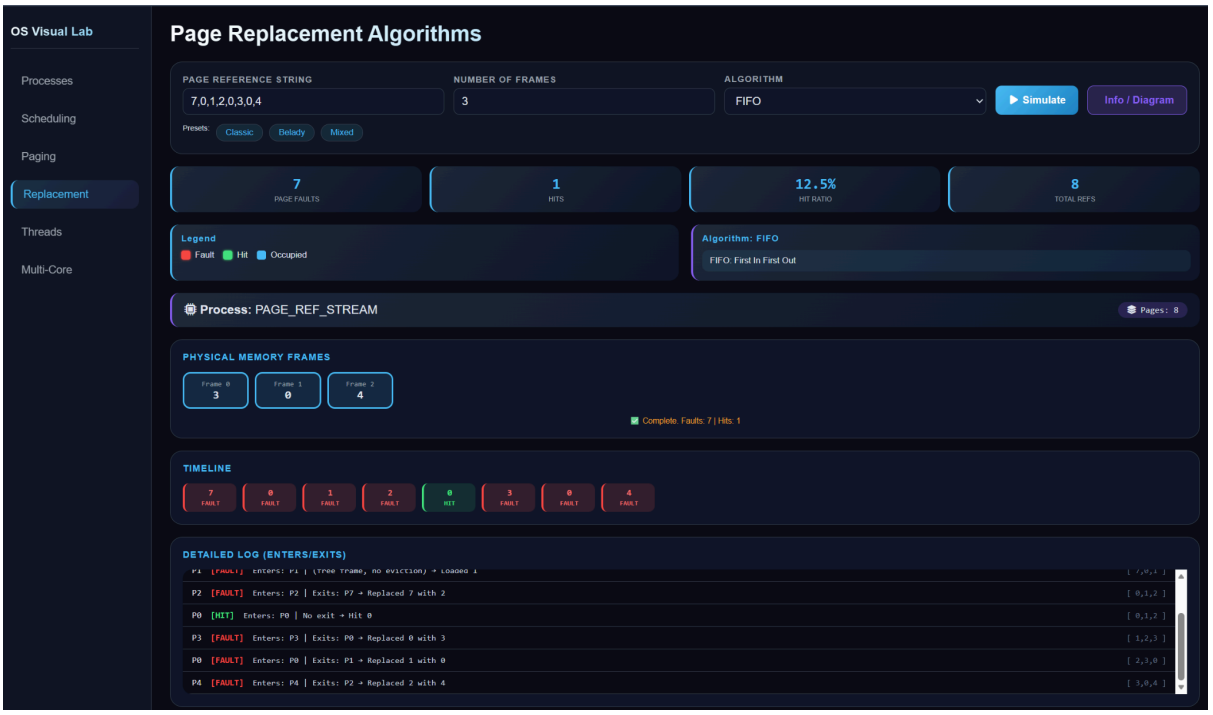
Hits	1
Hit Ratio	12.5%

Estado Final de los Frames (FIFO):

Frame	**Página**
Frame 0	3
Frame 1	0
Frame 2	4

Pruebas

Imagen 7. FIFO



Comentarios: Durante la simulación se observó que FIFO realizó reemplazos de páginas de manera constante, generando una alta cantidad de fallos de página. El algoritmo expulsó páginas únicamente por antigüedad, sin considerar si volverían a utilizarse en el corto plazo. El caso más representativo ocurrió cuando la página `0` fue reemplazada al ingresar la página `3`. Inmediatamente después, la página `0` volvió a ser solicitada, provocando un fallo adicional que pudo haberse evitado con una estrategia más eficiente.

Caso 2. LRU

Ejecución

Resultados Observados (LRU):

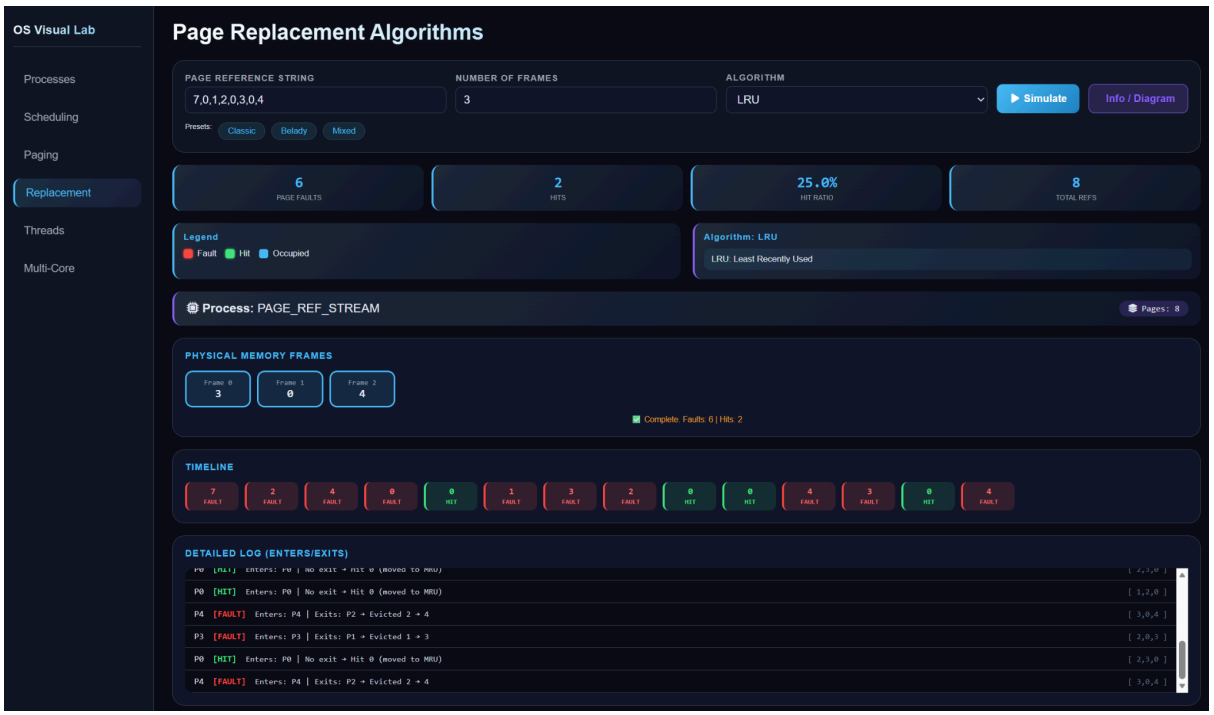
Métrica	Resultado
Page Faults	6
Hits	2
Hit Ratio	25.0%

Estado Final de los Frames (LRU):

Frame	Página
Frame 0	3
Frame 1	0
Frame 2	4

Pruebas

Imagen 8. LRU



Comentarios: El algoritmo LRU mostró un comportamiento más eficiente que FIFO al conservar en memoria páginas utilizadas recientemente. Esto permitió reducir la cantidad de fallos de página y aumentar el número de aciertos de memoria. Visualmente, la línea de tiempo del simulador mostró que LRU logró identificar reutilizaciones frecuentes de ciertas páginas, especialmente la página 0, evitando reemplazos innecesarios.

Caso 3. Óptimo

Ejecución

Resultados Observados

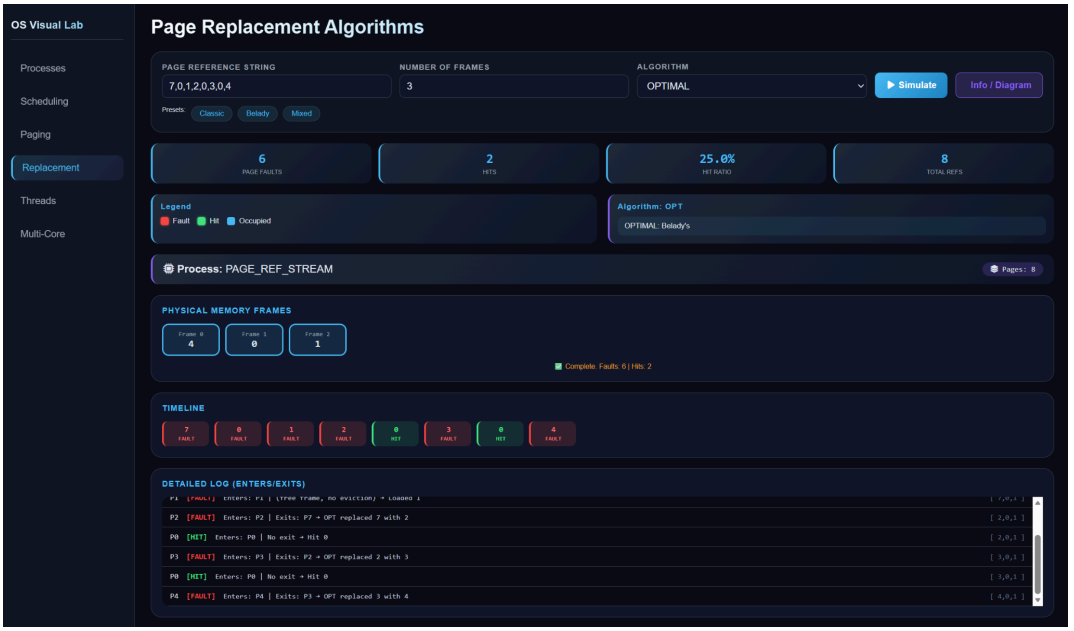
Métrica	Resultado
Page Faults	6
Hits	2
Hit Ratio	25.0%

Estado Final de los Frames (Óptimo)

Frame	Página
Frame 0	4
Frame 1	0
Frame 2	1

Pruebas

Imagen 9. Óptimo



Comentarios: El algoritmo Óptimo obtuvo el mejor desempeño teórico posible para esta secuencia de referencias. La simulación mostró que el algoritmo seleccionó estratégicamente las páginas menos necesarias a futuro, reduciendo reemplazos innecesarios. De manera similar a LRU, el algoritmo mantuvo en memoria páginas con alta probabilidad de reutilización inmediata, evitando fallos adicionales.

SetUp Threads

Objetivo: Analizar el comportamiento de distintos modelos de threading dentro del simulador, evaluando la ejecución concurrente de hilos, el uso del CPU, la capacidad de paralelismo y el impacto de los modelos Many-to-One, One-to-One y Many-to-Many sobre la administración de hilos.

Configuración del Escenario

- Módulo utilizado: threads.html
- Quantum utilizado: 3 U.T.
- Cantidad de hilos: 3
- Proceso asociado: P1
- Configuración de hilos:

Thread	**Arrival Time**	**Burst Time**
T1	0	5
T2	0	3
T3	0	4

Caso 1. Many-to-One

Ejecución

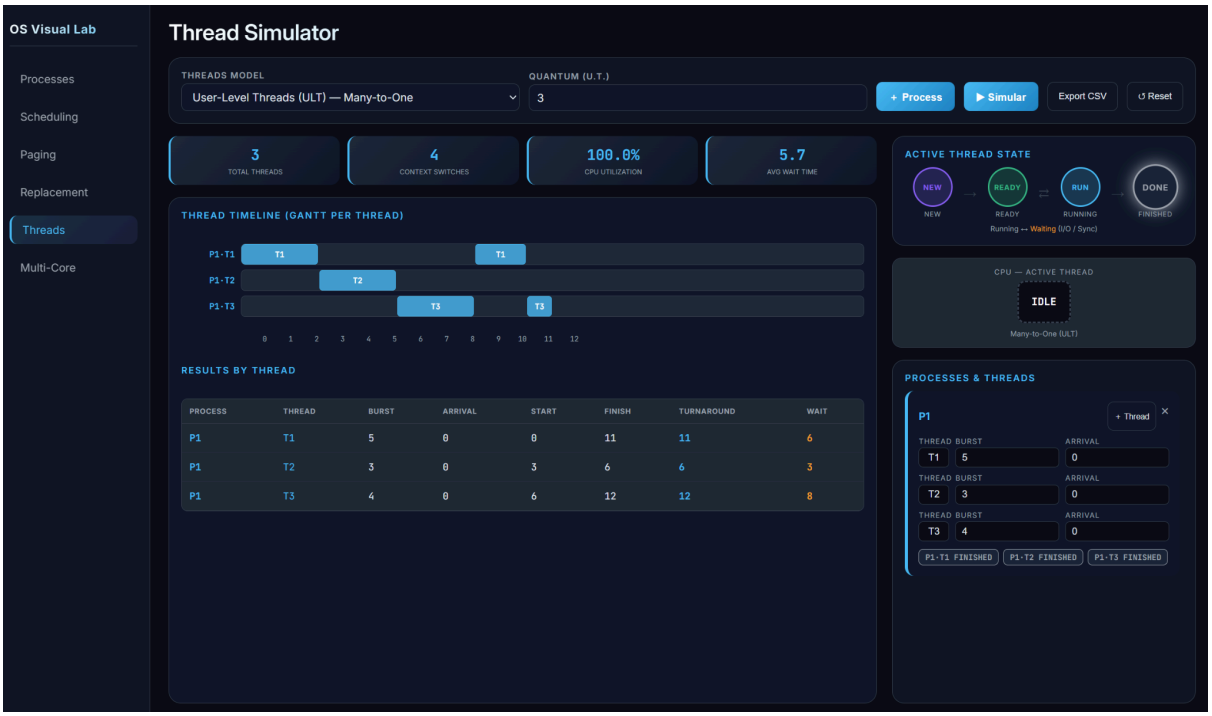
Resultados Observados:

Métrica	**Resultado**
Tiempo Total	12 U.T.
Ejercicio Paralela	No

Cores Utilizados	1
Tipo de ejecución	Concurrente Secuencial

Pruebas

Imagen 10. Many to One



Comentarios: Durante la simulación se observó una ejecución secuencial basada en Round Robin. Cada hilo recibió un quantum de 3 unidades de tiempo antes de ceder el CPU al siguiente hilo de la cola. En el diagrama de Gantt no se presentó solapamiento entre barras, indicando ausencia de paralelismo real. Únicamente un hilo permanecía en estado *RUN* en cada instante de tiempo. Asimismo, este modelo mostró una fuerte dependencia entre hilos. Conceptualmente, si uno de los hilos realizará una operación bloqueante de entrada/salida, todos los demás hilos quedarían detenidos debido a que el kernel administra una única entidad de ejecución.

Caso 2. One-to-One

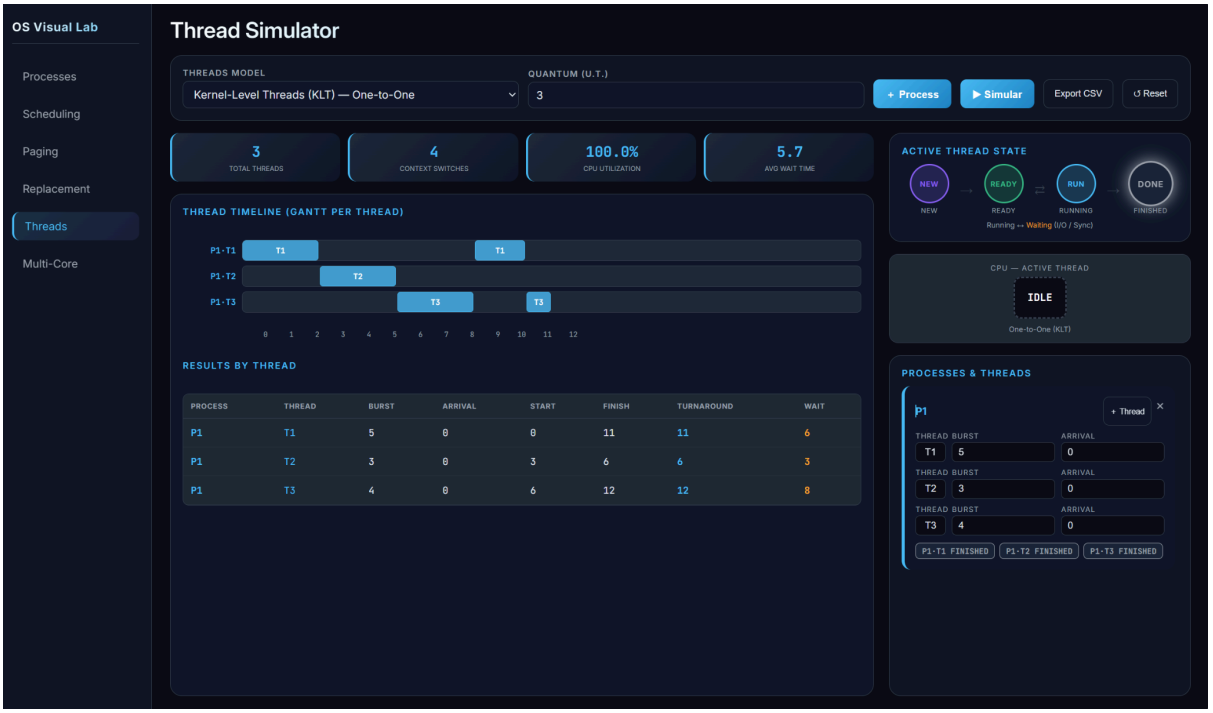
Ejecución

Resultados Observados:

Métrica	Resultado
Tiempo Total	12 U.T.
Ejercicio Paralela Visual	No
Capacidad de Paralelismo	Sí
Cores Utilizados	1

Pruebas

Imagen 11. One to One



Comentarios. Aunque en esta simulación específica no se observó paralelismo visual debido a la configuración del sistema y al uso de un único núcleo, el modelo One-to-One sí permite ejecución paralela real cuando existen múltiples cores disponibles. Visualmente, el comportamiento fue similar al modelo Many-to-One, mostrando ejecución intercalada sin

solapamiento de barras dentro del diagrama de Gantt. Sin embargo, el modelo presentó una diferencia importante en la administración de bloqueos. Si un hilo se bloquea, los demás pueden continuar ejecutándose normalmente, ya que el kernel controla cada hilo de manera independiente.

Caso 3. Many-to-Many

Ejecución

Resultados Observados:

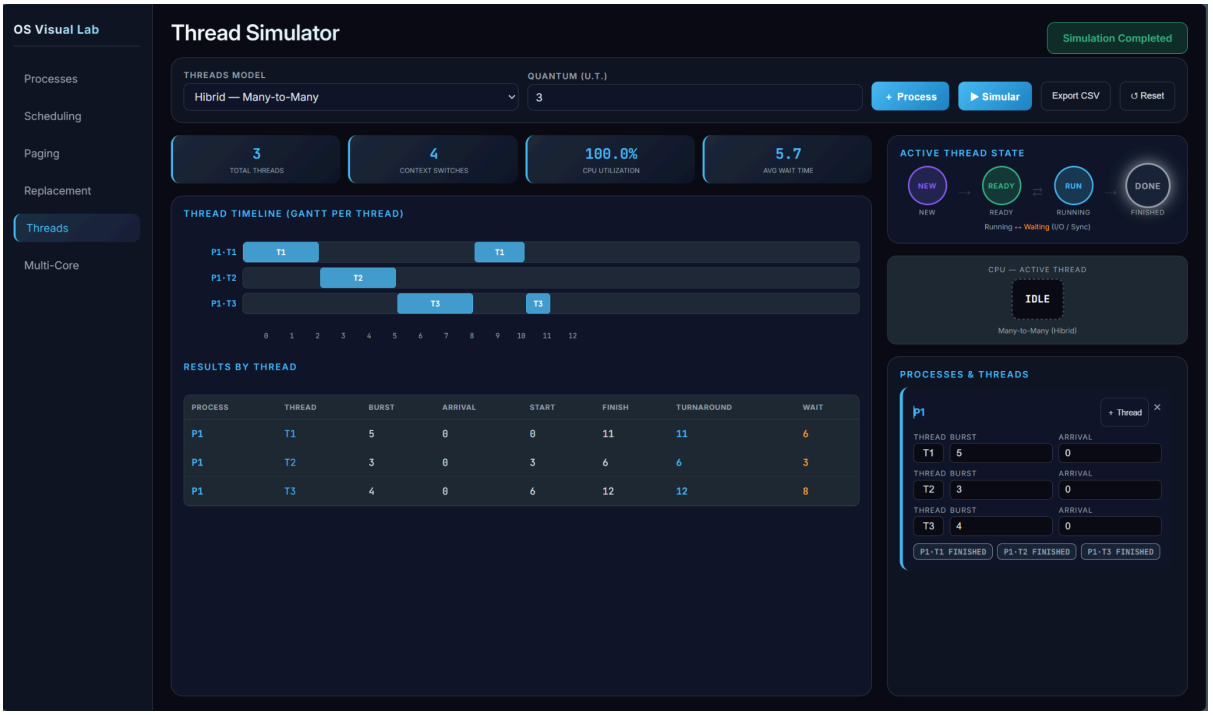
Métrica	**Resultado**
Tiempo Total	12 U.T.
Ejercicio Paralela Visual	No
Tipo de Ejecución	Multiplexada
Responsividad	Alta

Análisis: Aunque la cantidad de núcleos aumentó, el sistema no obtuvo beneficios adicionales en desempeño debido a que la carga de trabajo era insuficiente para aprovechar completamente los recursos disponibles.

La reducción en la utilización promedio del CPU evidencia la existencia de núcleos ociosos durante gran parte de la simulación. Esto demuestra que incrementar el número de cores no garantiza automáticamente mejoras de rendimiento si la cantidad de tareas concurrentes es limitada.

Pruebas

Imagen 12. Many to Many



Comentarios: Durante la simulación, el modelo Many-to-Many mostró un comportamiento híbrido. Aunque no se observó paralelismo visual dentro del escenario actual, el sistema presentó una administración más flexible de los hilos. El scheduler distribuyó dinámicamente la ejecución de los hilos, permitiendo mantener una mayor responsividad bajo carga. Además, este modelo puede evitar bloqueos globales al reasignar hilos de usuario hacia diferentes hilos del kernel. Conceptualmente, este modelo ofrece mayor escalabilidad en sistemas complejos con gran cantidad de hilos concurrentes.

SetUp Comparación de Algoritmos

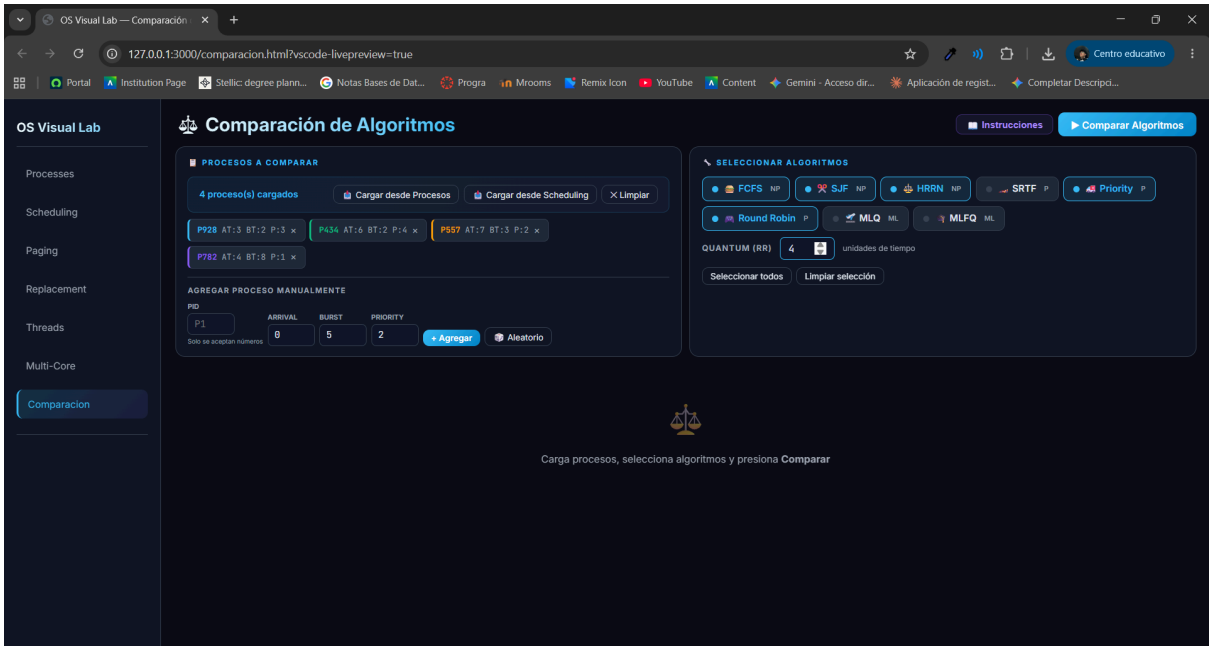
Objetivo: Analizar y comparar de manera simultánea el comportamiento y métricas de los distintos algoritmos de planificación abordados como FCFS, SJF, Round Robin, Priority y HRRN. Se analizarán sus métricas clave como tiempo de espera promedio, turnaround time, response time, la utilización del CPU, esto con el fin de identificar el algoritmo más eficiente con base en los procesos evaluados.

Configuración del Escenario

- Módulo utilizado: comparacion.html
- Cantidad de procesos: 5 procesos (P1 - P5)
- Algoritmos evaluados simultáneamente:
    - FCFS (First Come, First Served)
    - SJF (Shortest Job First)
    - Round Robin (Preemptive)
    - Priority (Preemptive)
    - HRRN (Highest Response Ratio Next)
- Quantum utilizado en Round Robin: 4 U.T.
- Variables observadas:
    - Avg Waiting Time
    - Avg Turnaround Time
    - Avg Response Time
    - CPU Utilization

Proceso	Arrival Time	Burst Time	Priority
P1	0	8	2
P2	1	4	3
P3	2	9	1
P4	3	5	2
P5	4	2	3

Imagen 1. Setup



Caso 1. FCFS vs SJF vs Round Robin vs Priority vs HRRN

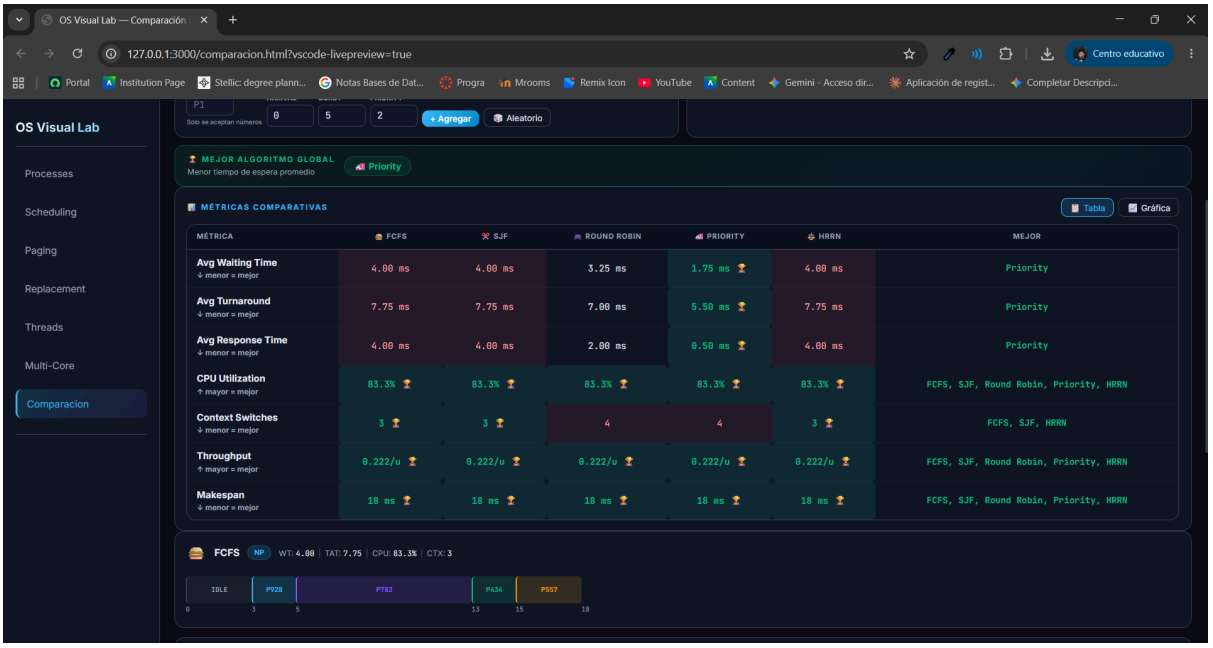
Ejecución

Resultados Observados:

Métrica	FCFS	SJF	Round Robin	Priority	HRRN	Mejor
Tiempo Espera Promedio	4.00 ms	4.00 ms	3.25 ms	1.75 ms	4.00 ms	PRIORITY
Tiempo de Turnaroun d Promedio	7.75 ms	7.75 ms	7.00 ms	5.50 ms	7.75 ms	PRIORITY
Tiempo de Respuesta Promedio	4.00 ms	4.00 ms	2.00 ms	0.05 ms	4.00 ms	PRIORITY
Utilización del CPU	83.3%	83.3%	83.3%	83.3%	83.3%	TODOS

Pruebas

Imagen 1. FCFS vs SJF vs Round Robin vs Priority vs HRRN



Comentarios: Al hacer la ejecución simultánea de los cinco algoritmos seleccionados con el mismo conjunto de procesos, el simulador permitió identificar las diferencias en desempeño de cada uno de ellos. Entre lo más relevante, se destacó que el algoritmo Priority fue el mejor algoritmo de manera general obteniendo los menores valores en tiempo de espera promedio, turnaround y tiempo de respuesta. Esto se debe a que el algoritmo se caracteriza por asignar el CPU al proceso con base en su prioridad asignada, mostrando una clara ventaja frente al resto de algoritmos.